AI Native Software Engineer

Finding problems is my job.

Technology is
a means to
solve them.

Garam Kim

kimgalam0831@gmail.com
github.com/kgr0831

Garam Kim

AI Native Software Engineer

I find the problem first, then choose how to solve it.

kimgalam0831@gmail.com
github.com/kgr0831
linkedin.com/in/kimgalam0831
kgr0831.tistory.com

How I approach problems

I find problems in the friction of everyday work and production. Technology is a means to solve them.

I draw on the CS foundations I built at Jungle and learn with AI, then verify that the original problem is resolved.

MOAI · AX Engineer
Intern · Sep 2026 - Present

Development tools

Unity
Next.js

AI tools

Orca
Codex
Hermes

Website ↗
kgr0831.github.io/

Experience built through solving problems

Work & research

Sep 2026 - Present

MOAI

Intern · AX Engineer

AI usage ledger · Admin tools · Leadbot

2026.04 – 2026.08

Soongsil Institute for Peace and Unification

SmartAI Lab · Undergraduate researcher

Pyongsong social simulation research

2026 KOSSCCHTHON

Grand Prize · 1st place

Kookmin · Soongsil · Soonchunhyang

Joint university competition

Education & training

Mar 2026 - Present

Soongsil University

School of AI Software

2025.09 – 2026.01

KRAFTON Jungle

SW-AI Lab, 11th cohort

PintOS · EUM

2023.03 – 2026.02

Korea Game Science High School

Computer Game Development

Student council VP and president / Founded and led Dash

The problems I have worked on

01	**Repetitive work and disconnected tools**	MOAI · Gammaru · Abstraction Bot Screen-golf CMS · PaperBridge	05–14 ↗
02	**Quality issues that get in the way**	EUM · PintOS Pyongsong social simulation	15–20 ↗
03	**Limited time and production capacity**	Dudri · Spell Thrower · PlanetGame · Triniacle SpriteAIMaker · Remnents of Light / ROLAI	21–36 ↗
04	**Conversation and new service needs**	Virtual Friend · Orca-STS Character memory and voice development tools	37–42 ↗

Automating work for companies, clubs, and myself

Context	Project	Problem identified	Automation scope
Company	MOAI usage ledger	Scattered AI usage records	Collect, record, and review usage
Company	MOAI admin tools · Leadbot	Task lookup and follow-up spread across tools	Admin APIs and chat-based work
Club · Personal	Gammaru Web · Daily Brief	Repeated research and information sharing	Research → Report → Archive and notify
Club · Team	Abstraction Bot	Discussions and decisions buried in chat	Collect → Summarize → Record
Business	Unattended screen-golf CMS	Separate booking, devices, payments, and settlement	Operations and ledger-based settlement
Researchers	PaperBridge	Switching between papers and AI tools	Select context → Ask → Check evidence

MOAI · Work automation and admin tools

MOAI · AX Engineer Intern · Sep 2026 - Present

Operators needed to review scattered AI usage and task records, and turn chat requests into follow-up work.

AI usage ledger

Bring usage records together

Contributed to the ledger that collects, records, and presents usage across AI tools.

Admin interface

Align filters and task counts

Implemented search, filters, and pagination, and aligned summary counts with actual task status.

Leadbot · In development

From chat to follow-up work

Contributing to a bot that connects web chat and Google Chat requests to work tools.

What changed

Operators can distinguish overall task status from the current filtered list, even after changing filters or loading more results.

Repository · Private ↗
github.com/moai-technologies/moai

Admin implementation ↗
github.com/moai-technologies/moai/pull/898

Challenge · Keeping counts and results consistent

When users changed filters quickly, an older response could arrive late and overwrite the current results.

User selection	**Previous filter request**	→	**Current filter request**	→ Keep the current filter
Response handling	**Apply current response**	→	**Older response arrives**	→ **Ignore stale results**

Aggregation

Separate totals from the list

Count all tasks, incomplete tasks, tasks due today, and overdue tasks across the full dataset.

Classification

Use the same status rules

Include in-progress tasks consistently in incomplete and overdue counts.

Exceptions

Identify why a request failed

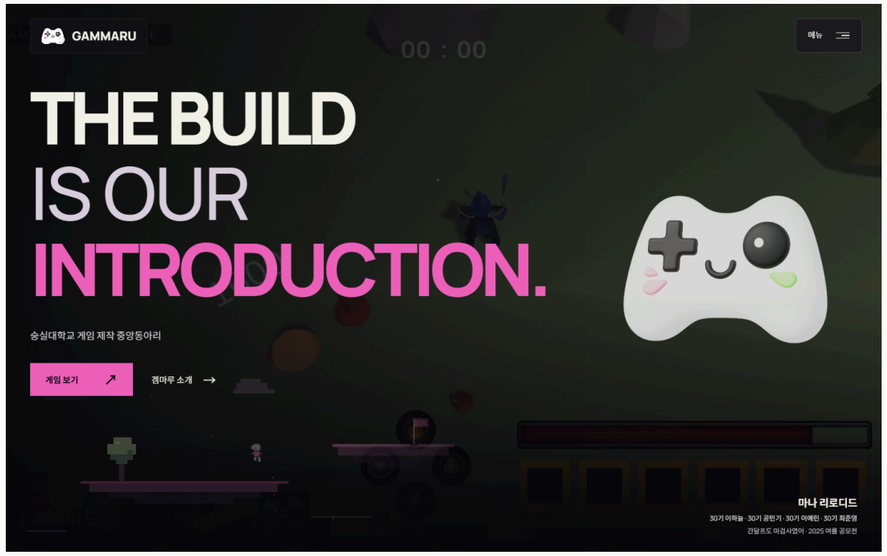
Distinguish invalid dates, missing task-target data, and processing failures.

Result · Consistent search results

Late responses no longer replace the current filter selection. Summary counts and task lists use the same classification rules.

Search implementation · Private ↗
github.com/moai-technologies/moai/pull/898

Gammaru Web · Daily Brief



Gammaru Web · Club introduction and game archive

Club website ↗ gammaru.dev/

Report archive ↗ d3ktbkaahtexzk.cloudfront.net/reports

Problem identified

Finding and sharing the same information daily

The Gammaru game development club at Soongsil repeatedly researched sponsorship and funding. I connected a report archive to the club website.

My implementation

Automate delivery after the research

Built date-based archives, search, Discord login, subscription approval, and notifications. Personal competition and job briefs use separate audiences and purposes.

Separate public pages and member reports

Published the club and game archive on GitHub Pages; login and reports remain on AWS.

Website code ↗
github.com/kgr0831/Gammarupage

Daily Brief code ↗
github.com/kgr0831/DailyScrum_Bundle

Challenge · Repeated execution and operating cost

Brief publication workflow

Research
Funding and competitions

→

Write report
HTML for each purpose

→

Archive & access
Dates and subscriptions

→

Notify Discord
Approved recipients

Issue found in operation	Decision and fix	What changed
Repeated approval for the same channel	Distinguish a failed name lookup from a changed recipient	Avoid repeated checks when the approved channel ID is unchanged
Status checks increased storage requests	Reduce polling and document recovery steps	Account for operating cost after publication and delivery
Public and member content need different access	Public pages on GitHub Pages; reports on AWS	Published the homepage while retaining login, schedules, and notifications

Recipient check fix ↗
github.com/kgr0831/DailyScrum_Bundle/commit/9b78be8

Polling and recovery improvements ↗
github.com/kgr0831/DailyScrum_Bundle/commit/1acc6dc

Abstraction Bot · Turn conversations into records

Built a tool that summarizes Discord discussions and records work, making decisions and action items easier to find again.

Discord → **Conversation store** → **Connected AI tool** → **Notion · Discord**

Collect messages | Save and retrieve messages | Summaries and daily reviews | Organize and share results

Challenge · Changing the summarization method

Separate collection from summarization

Decoupled the collection bot from the summarizing agent so collection continues when the AI method changes. Summaries run through the user's connected AI tool.

Implementation result

Manage collection and reviews in a web console

Provided scheduled and manual reviews, connection status, and message-retention controls.

Challenge · Distinguish conversation from commands

Restricted tool use so chat cannot act as commands. The bot provides statistics without AI and separate message-retention controls.

Code and architecture ↗

github.com/kgr0831/Abstraction-Bot

Unattended screen golf · CMS and settlement

A web system for bookings, device control, payments, and settlement

Problem identified

Bookings, devices, and payments operated separately

Operators need to align reservations with device power and trace payments and refunds through to settlement statements.

Payments & refunds
Collect transactions

→

Settlement ledger
Record each transaction

→

Settlement & reconciliation
Aggregate and compare

> Centralized settlement calculations so **the interface and totals use the same ledger**.

Result · Connected operations and settlement

- Store, device, booking, and member management
- Jobs to reconcile bookings with device state
- Duplicate, refunded, and late-arriving transactions
- Finalization, reconciliation differences, and audit records

CMS built with Next.js

Operational screens and settlement totals use the same ledger.

Implemented and demonstrated management and settlement features. Payment-provider and physical-device integrations are a separate scope.

Implementation · Private ↗
github.com/kgr0831/golf-cms

Challenge · Trace the basis of settlement totals

Changing only a total obscures how refunds, duplicate notices, and late transactions affected the amount.

Transaction ledger
Sales, discounts, points, fees

→

Aggregate & finalize
Generate and lock statements

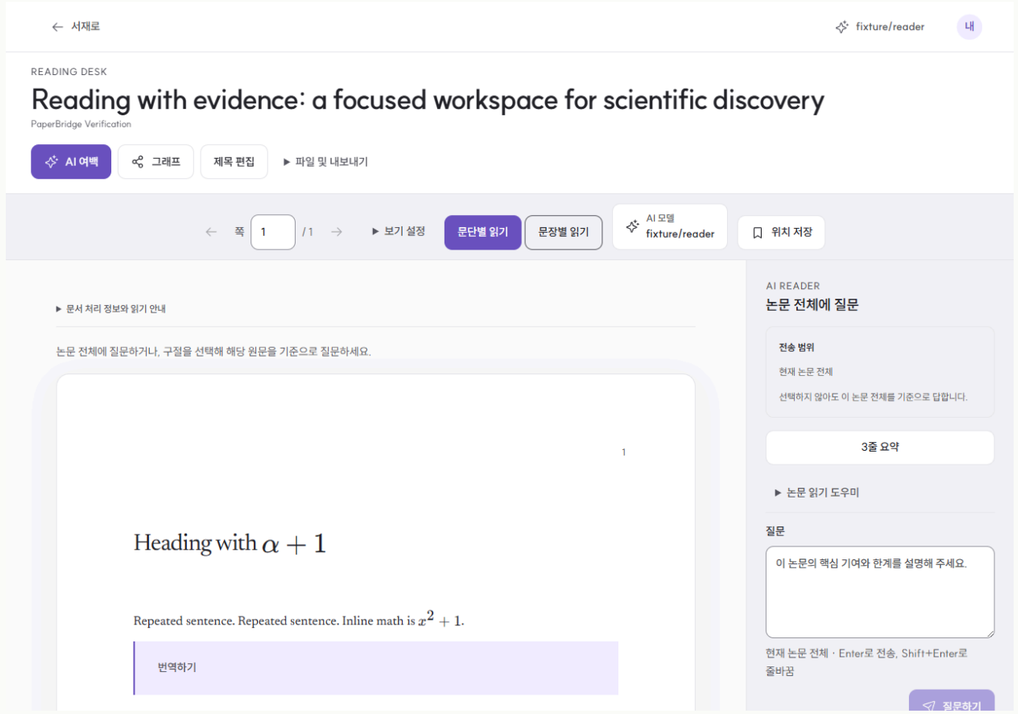
→

Reconcile & recalculate
Compare with external totals

Situation	Handling	Evidence retained
Refund	Append a ledger entry with the opposite sign	Original transaction and refund history
Duplicate notice	Deduplicate by transaction ID	Prevent counting the same transaction twice
Arrival after closing	Keep the finalized statement and carry forward	Transaction included in the next period

Implemented and verified management, settlement, and exception handling. Payment-provider and physical-device connections remain outside this implementation.

PaperBridge · Read papers and keep evidence



PaperBridge reader verification with a sample document

Problem identified

Switching between a paper and a chat tool

Repeatedly copying passages, explaining the context again, and locating evidence in the source.

Solution · My contribution

AI connection, reader interactions, and backend

Connected AI tools, selected passages, and questions within the reader. Contributed to authentication and research-workspace backend features.

> **Result · Ask and check evidence inside the reader**
> Select passage → Ask AI → Compare answer and source

Project introduction ↗
github.com/kgr0831/PaperBridge

Contribution code · Private ↗
github.com/myunhh/PaperBridge-Refectored

Challenge · Connect the reader to AI tools

Reduced the need to explain paper context again whenever a reader switched to an external AI tool.

Read a paper
Select a source passage

→

Pass the context
Selection and question

→

Run the AI tool
Check connection and settings

→

Check evidence
Compare answer and source

Technical challenge

AI tools installed in different locations
Search the configured path, system search paths, and default install locations in order. Distinguish the causes of connection failures.

Design decision

Separate connection checks from execution
Check tool availability first, then let the user review the question, context, and model before running it.

My contribution
Contributed to AI connections, reader interactions, authentication, and the research-workspace backend, keeping questions and evidence within the reading flow.

My contribution · Private ↗
github.com/myunhh/PaperBridge-Refectored/commit/03f551b

Project introduction ↗
github.com/kgr0831/PaperBridge

EUM · AI collaboration workspace



Service screen: video meetings with live AI translation

Project presentation ↗ youtube.com/watch?v=Pnjw9WitAXg

Problem identified

Drawing during meetings made the screen sluggish

A collaboration service combining AI-translated video meetings and a shared whiteboard. I identified latency and drawing-quality issues during concurrent use.

Role · Five-person team

Whiteboard full-stack developer

Dec 2025 - Jan 2026. Owned the drawing renderer, live synchronization, sticky notes, permissions, and other whiteboard features.

Approach

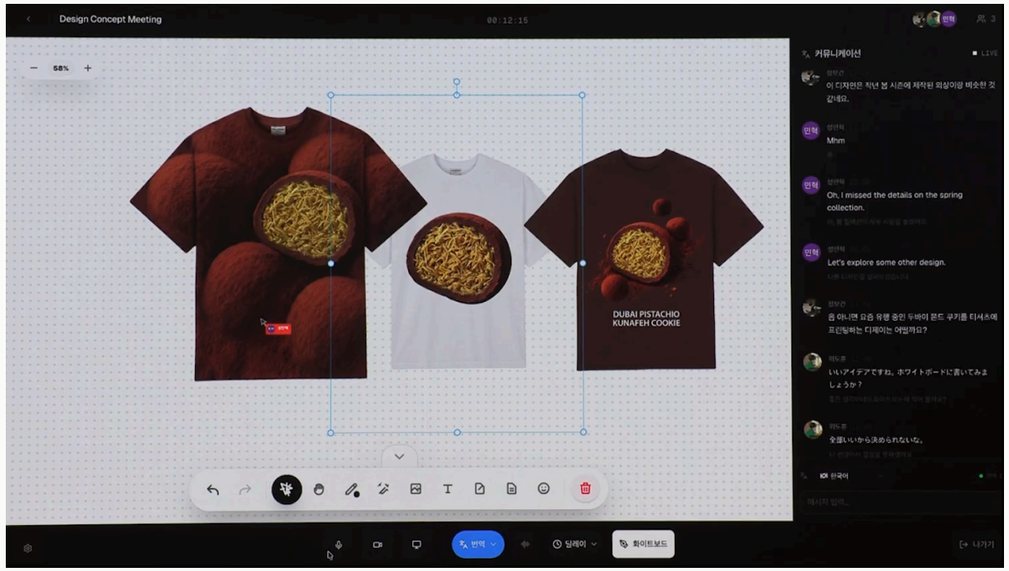
Separated rendering load, rough strokes, and transmission latency, then chose a method for each cause.

Team code · Private ↗
github.com/jb0xyz/EUM

Presentation video ↗
youtube.com/watch?v=Pnjw9WitAXg

Jungle project introduction ↗
jungle.krafton.com/news/103

EUM · Resolve the whiteboard rendering bottleneck



EUM · Shared real-time whiteboard

Result · Average frame rate

23.7 → 58.7 FPS

Migrated to WebGPU and PixiJS v8
Measured during the project

Problem identified

Slow response when drawing during a call

Video calls and whiteboard work competed on the same execution thread. More shapes increased CPU load and slowed drawing and screen response.

Chosen approach

Move graphics work to the GPU

Migrated to PixiJS v8 with WebGPU to reduce CPU-bound rendering. Learned unfamiliar graphics concepts with AI and explained the migration to the team.

Five-person team · Whiteboard developer · Dec 2025 - Jan 2026

Code ↗
github.com/kgr0831/EUM2

Presentation video ↗
youtube.com/watch?v=Pnjw9WitAXg

EUM · Improve strokes and reduce transfer volume

Separate fixes for input, transfer, and display

One Euro Filter

Reduce input jitter

Douglas–Peucker

Keep essential points

Bézier Curve

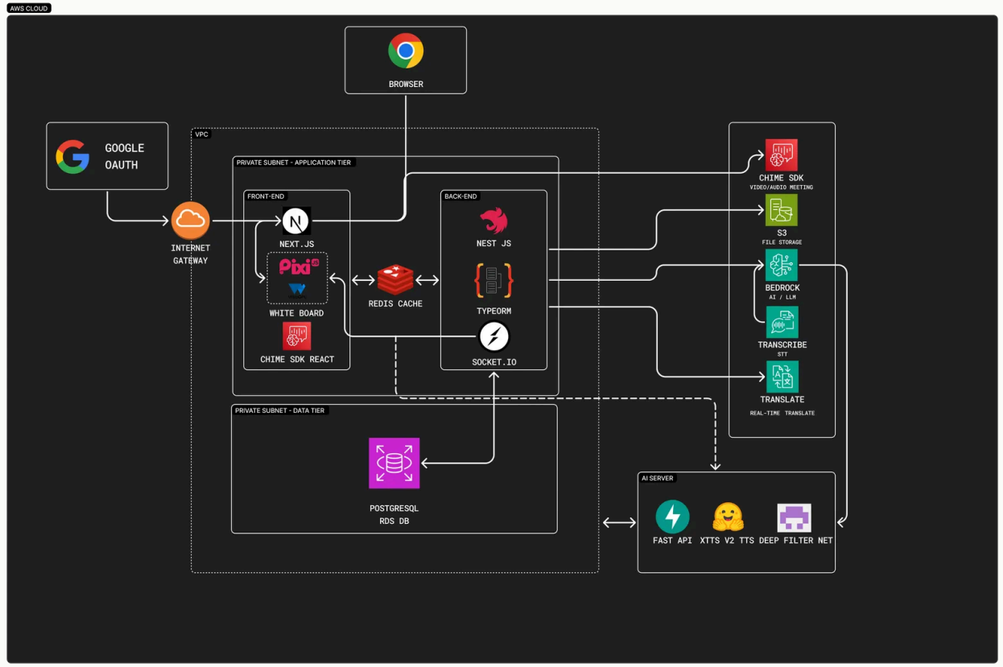
Interpolate smooth curves

User-facing issue	Method	Improvement
Rough and shaky strokes	Adjust filtering to drawing speed	Reduce coordinate jitter while retaining responsiveness
High transfer volume and sync latency	Keep the points that preserve the shape	Reduce the data sent
Angular strokes when drawing quickly	Join points with curves	Smooth visibly segmented lines

Chose algorithms for stroke quality and latency. WebGL provides a fallback when WebGPU is unavailable. Concurrent edits keep the last change.

Implementation ↗
github.com/kgr0831/EUM2

EUM · Align the team on an architecture change



EUM service architecture from the original project

Problem encountered

Team agreement on a structural change

Replacing the renderer affected the schedule and existing features. I explained the cause and proposed migration in documents and presentations to align the team.

Trade-off

Performance, compatibility, and concurrent edits

Used WebGL as a fallback. Batched coordinates every 50 ms and compressed transmission. Concurrent edits keep the last change; automatic conflict merging is not implemented.

Selection criteria

Considered compatibility and the team's adoption cost alongside performance.

Whiteboard implementation · Private ↗
github.com/jb0xyz/EUM/pull/27

Presentation video ↗
youtube.com/watch?v=Pnjw9WitAXg

PintOS · Finding the flaw in my own hypothesis

KRAFTON Jungle · Four-person team · Scheduling, user programs, and virtual memory

Addressed priority inversion: a high-priority task waits for a lower-priority task to release a resource.

Hypothesis · Release-Trigger experiment

Reduce the cost of tracking dependencies

Added a release_trigger structure to each thread and batched lock release and priority restoration at scheduling time, as an experiment to replace donation lists.

The problem I discovered

Donation is state, not a one-off event

Multiple and nested donations change dependencies per lock. A single trigger lost that state. Delaying unlock until scheduling also caused deadlocks and lost priorities.

Track donations per lock

Handle multiple and nested dependencies

→

Recompute on release

Remove only the released lock's donations

→

Test multiple and nested donation

Resolve races and pass the tests

Lesson · Find what a simplification leaves out

Refactored to standard priority scheduling. Tracing the failure showed why persistent state and immediate lock-release handling are essential to the design.

Project · Private ↗
github.com/kgr0831/KAIST_PintOS

Pyongsong social simulation · AI research

Research question

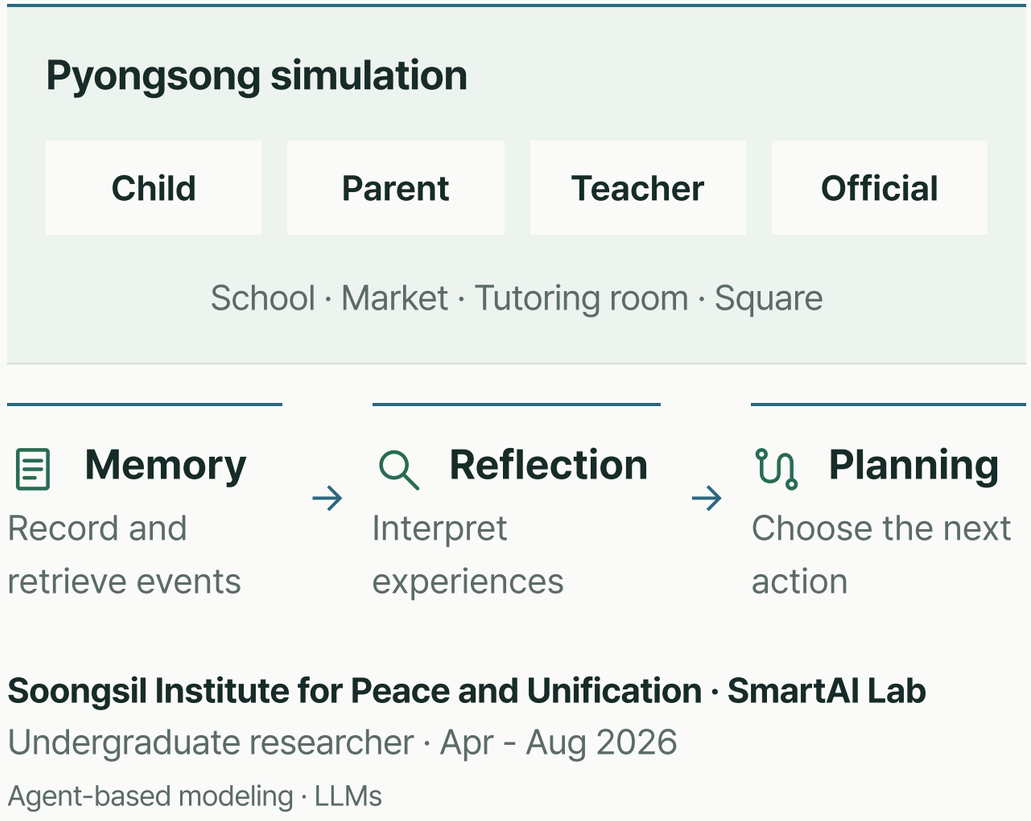
How do different environments and memories shape behavior?

Contributed to a research simulation of Pyongsong, where direct observation is difficult. Virtual people interact and act according to their memories and situations.

Contribution · Initial simulation library

Implemented shared agent, space, and world structures; memory, reflection, and planning loops; interviews; and event and memory exports.

Connected memory, reflection, and planning, using interviews and event records to trace behavior. Research-model results are not treated as predictions of the real society.



Soongsil Institute for Peace and Unification · SmartAI Lab

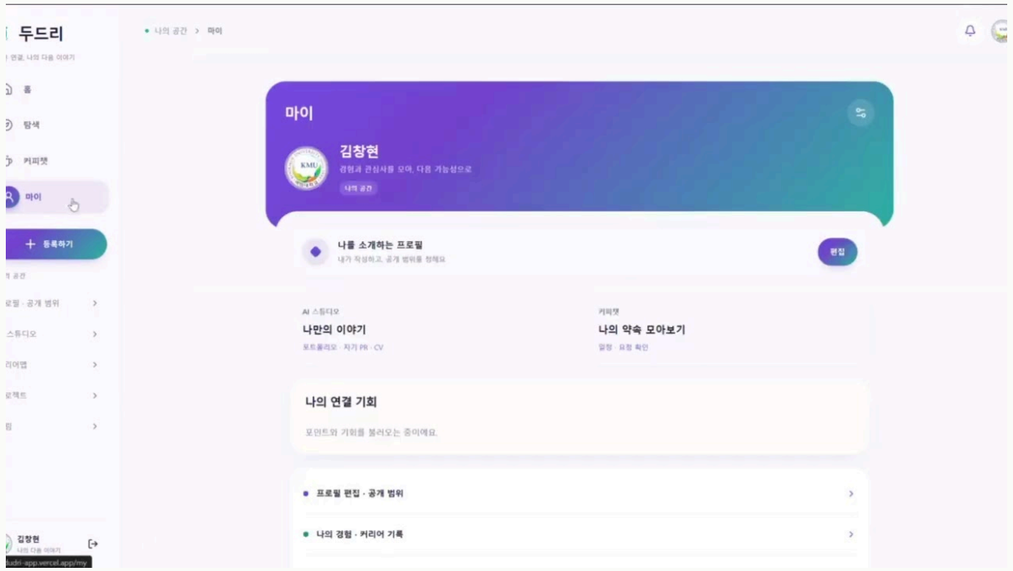
Undergraduate researcher · Apr - Aug 2026

Agent-based modeling · LLMs

Research code · Private ↗

github.com/kgr0831/SimulationProject

Dudri · Connect experience with opportunity



Thumbnail from the Dudri introduction video

Service introduction video ↗ youtube.com/watch?v=vvsyQlOtiUQ

Problem identified

Turning past experience into the next opportunity

A platform for university students to organize experiences and connect with peers. It combines AI-generated application documents, coffee chats, and project participation.

Development experience

From planning to implementation in a hackathon

Contributed through an AI workflow linking specifications, implementation, and verification. After the award, continued improving generated-data consistency and editing.

Result

Grand Prize (1st place), 2026 KOSSCCHTHON: a joint competition of Kookmin, Soongsil, and Soonchunhyang universities.

Project code ↗
github.com/kgr0831/KOSSCCHTHON_2026

Dudri · An AI-native workflow and Grand Prize



2026 KOSSCCHTHON Grand Prize award photo

Grand Prize · 1st place

Kookmin · Soongsil · Soonchunhyang 2026 joint university competition KOSSCCHTHON

Connected specification writing, feature implementation, and verification with AI to turn a plan into a service within the hackathon schedule.

Specify
Define concrete tasks

→

Develop
Implement each feature

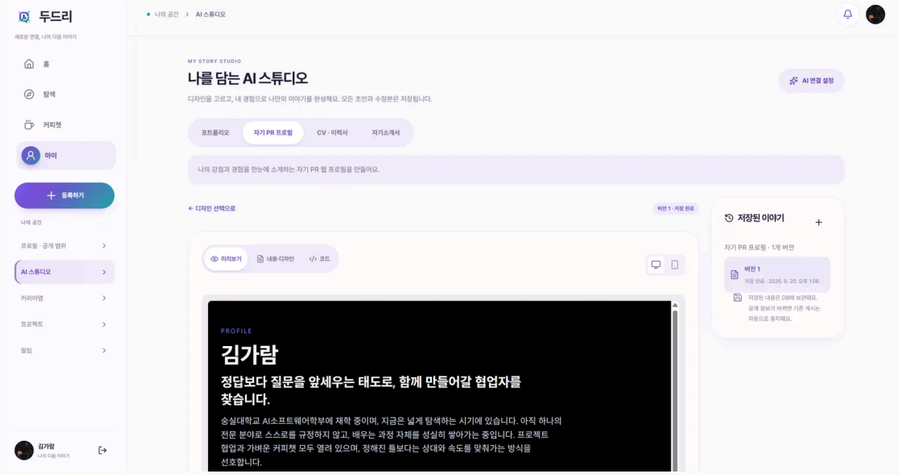
→

Verify
Check behavior and revise

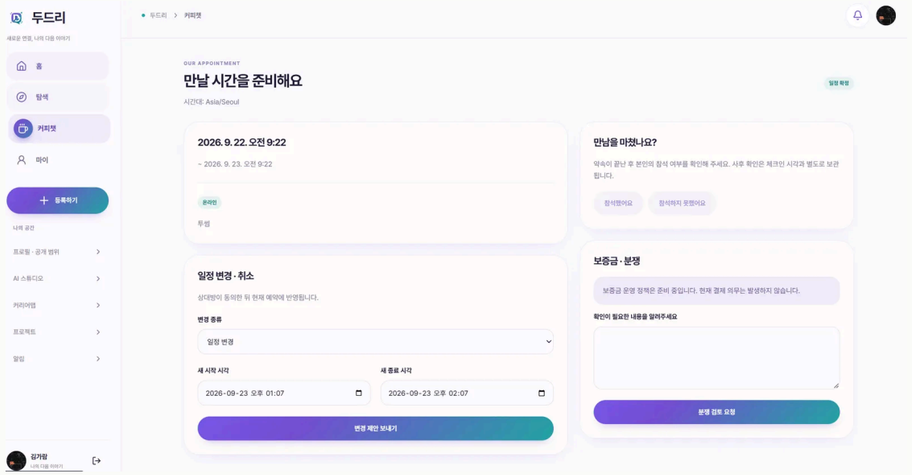
Continued improving generated-data consistency and user editing after the competition.

Code ↗
github.com/kgr0831/KOSSCCHTHON_2026

Dudri · From experience to documents and projects



AI Studio · Create application documents from experience



Coffee-chat scheduling · Share experience with peers

User workflow

Select experience and edit the document

Connected source selection, AI generation, user editing, and preview.

User workflow

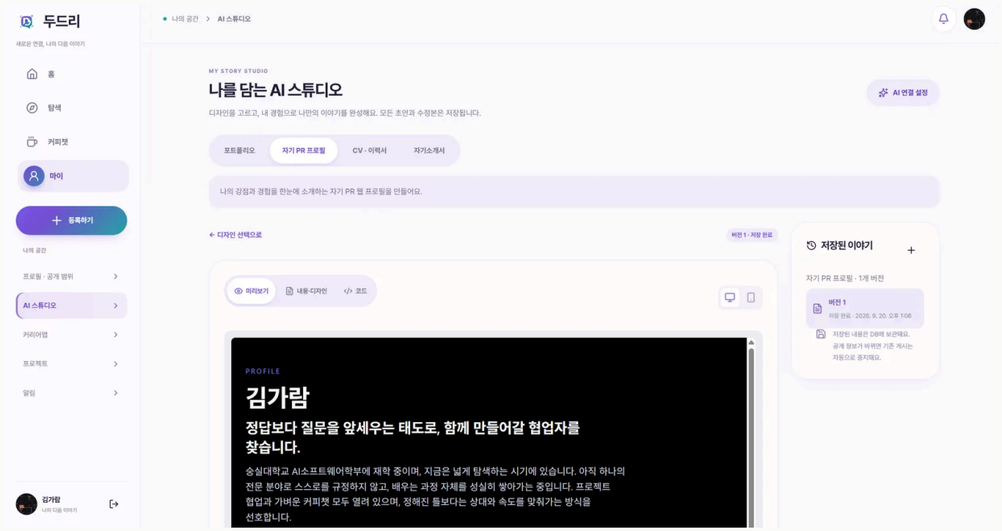
Connect to the next experience through a chat

Request a meeting with another student and coordinate a time.

Service introduction video ↗
youtube.com/watch?v=vvsyQlOtiUQ

Code ↗
github.com/kgr0831/KOSSCCHTHON_2026

Dudri · Give users control over AI-generated work



Dudri AI Studio · Includes development after the hackathon

Design principle · User control

Users choose source material, review AI suggestions, and decide what to edit and publish.

Problem and solution · Generated-data consistency

Source edits can leave an outdated generated result

Separated generation jobs from document versions. Check for source changes before saving and block publication of results based on outdated material.

Problem and solution · Visual and code editing

Visual edits could erase code written by the user

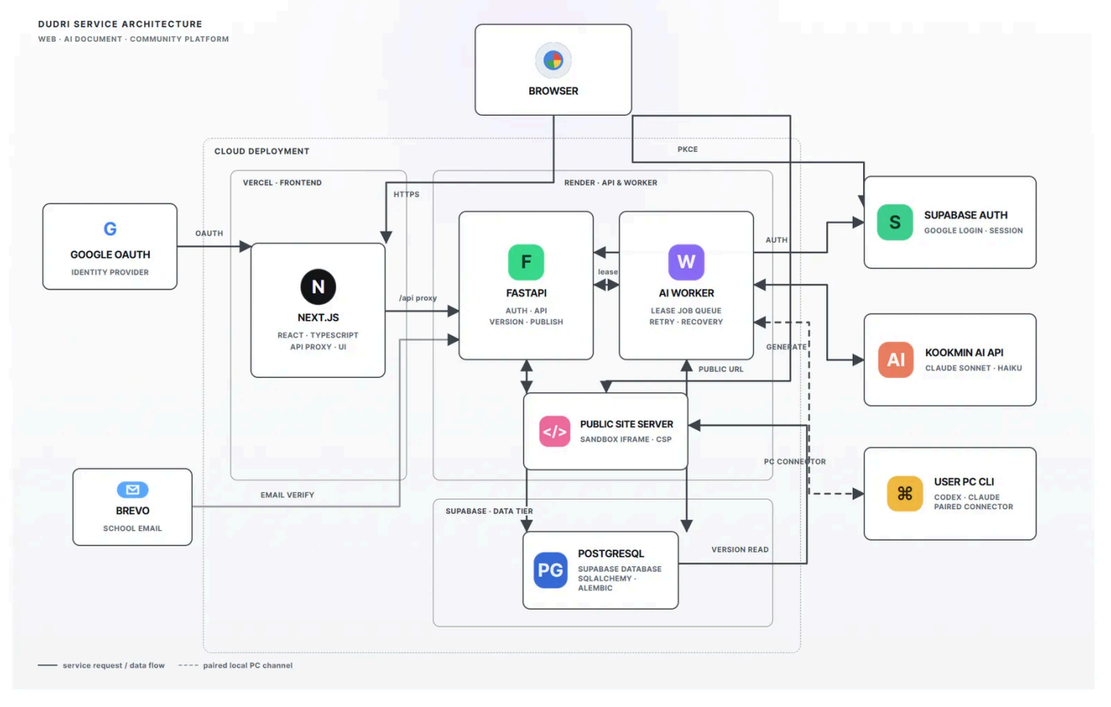
Assigned identifiers to editable regions and updated only changed parts. Preserve user code and layout, and save edits as new versions.

Generated code runs in an isolated view with restricted resource access.

Code and design ↗

github.com/kgr0831/KOSSCCHTHON_2026

Dudri · Asynchronous work and user control



Dudri architecture · Generation and publication flow

Design experience

Separate generation, editing, and publication

Separated states so users can edit source data or cancel during generation. Set deadlines for background jobs and designed interrupted work to be retried.

Trade-off

Editing freedom and controlled publication

Support visual and code editing while isolating generated-code execution and restricting access. Users decide whether to edit and publish.

Remaining limitation

Free-server cold starts still delay the first response. Saved job records allow recovery after interruption.

Code and design ↗
github.com/kgr0831/KOSSCCHTHON_2026

Improving the way I build games

Project	Work involved	Workflow development
Spell Thrower	Game jam · Multiplayer · Matchmaking	Build small features and integrate with the team
PlanetGame	Galaxy, camera, tutorials, and cinematics	Connect multiple systems into a coherent play flow
Unitrio / Triniacle	Combat, boss AI, crafting, and tutorials	Divide features, review code, and integrate
SpriteAIMaker	Characters, directional animation, and sheets	Connect game-asset production in one tool
Remnents of Light / ROLAI	Development requests, Unity execution, and review	Automate remote requests through validation and reporting

My scope grew across collaboration, system implementation, asset production, and verification.

Spell Thrower · A 1v1 multiplayer game

During a 1v1 test, two game instances shared the same authentication data and could not connect as separate players.

Client A

Authentication profile per process

Client B

Independent player identity

→

Matchmaker · Relay

Session entry and connection

→

1v1 play

Integrate with the team's gameplay

Issue found in testing	Implementation
Two builds on one PC shared an identity	Use process IDs to separate anonymous profiles
Retry after failed initialization or cancellation	Recreate setup tasks and clear session references on exit

Result and remaining work

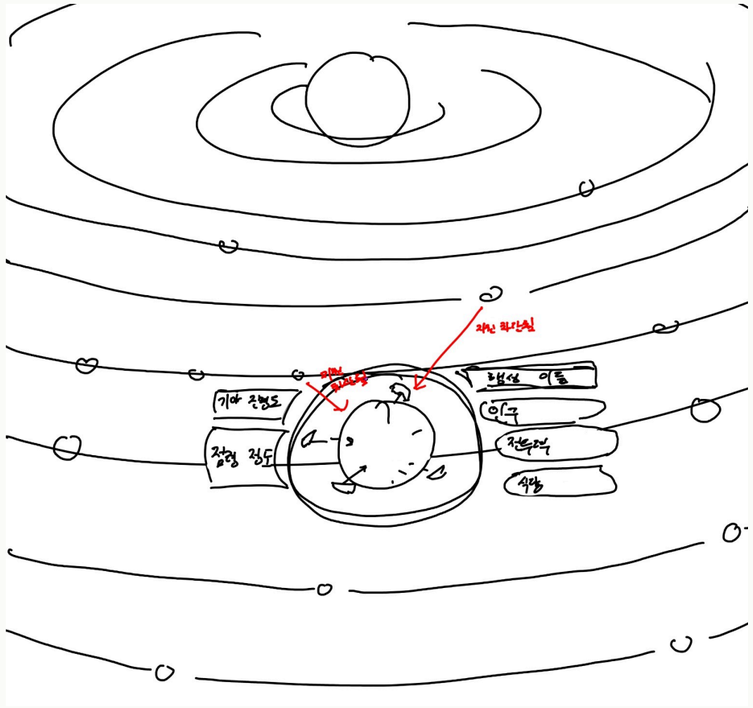
Integrated connection, matchmaking, and 1v1 play into the team's game. Communication-module conflicts bypassed in the prototype still need resolution, and encrypted transport must be restored before release.

Multiplayer code · Private ↗
github.com/kgr0831/GameJam2026/pull/1

Matchmaking code · Private ↗
github.com/kgr0831/GameJam2026/pull/2

PlanetGame · A playable flow for space strategy

A Unity 3D strategy game with galaxy exploration, resources, production, conquest, and research



Early planet and UI concept preserved in the repository

Problem identified

Using a menu also moved the camera

UI and camera inputs conflicted, while players had to learn several systems. Blocked camera input during UI interactions.

My implementation

Galaxy expansion, camera, tutorials, and cinematics

Connected navigation, controls, rule learning, and event presentation. Tutorials introduce one action at a time, and cutscenes pause game progression.

Implementation result

Made complex rules approachable through interactive tutorials and scene transitions, with skip and replay support.

Project code ↗
github.com/kgr0831/PlanetGame

Feature implementation ↗
github.com/kgr0831/PlanetGame/commit/d1c2f64

Unitrio / Triniacle · A team-built 2D action RPG

Five-person team · Team leader · Unity



Boss attacks overlapped with the next action.

A 2D action RPG built around combat and exploration. As team leader, divided features and integrated my combat, monster behavior, and tutorial work.

Game features I built

Code ↗	**Boss behavior** Attack and chase conditions in a behavior tree
Code ↗	**Neutral monster AI** Combat-target reactions and behavior
Code ↗	**Combat skill system** Expand the player's action choices
Code ↗	**Tutorial** Introduce the game's first interactions

Code and contributions ↗
github.com/kgr0831/UniTrio-Game-2026

Triniacle · Fix the order of boss skill execution

Putting cooldown before a skill let short skills overlap the next action and animation, causing movement jitter.

Previous sequence

Cooldown → Execute skill → **Overlap the next action**

Revised sequence

Telegraph → Animate and attack → **Cooldown**

Check distance
Determine attack eligibility

→

Choose behavior
Select a skill or chase

→

Execute skill
Telegraph → Animate → Attack

My contribution · Team leader

Moved cooldown after the attack to prevent overlapping actions. Implemented boss and monster behavior, combat skills, and tutorials, and integrated them into the team's game.

Boss behavior code ↗
github.com/kgr0831/UniTrio-Game-2026/pull/11

Combat skill code ↗
github.com/kgr0831/UniTrio-Game-2026/pull/17

SpriteAIMaker · AI asset production for games

Using generated images in a game also requires consistent direction, motion, frame size, and alignment.

Review character

Approve the reference image

→

Generate animation

Motion for each direction

→

Normalize & edit

Frame position and size

→

Export to Unity

Assemble and slice sheets

Workflow change

Generate characters and animation separately

Approve the reference character before generating motion, then continue with pixel editing and sheet assembly.

Technical challenge

Manage quality after generation

Fixed duplicate frames, misalignment, and color damage. Assemble directional results correctly even when generation finishes out of order.

Implementation result

Connected AI image generation and pixel editing in one tool, from character review through frame processing to game-ready export.

Code ↗
github.com/kgr0831/SpriteAIMaker

Change history ↗
github.com/kgr0831/SpriteAIMaker/blob/master/CHANGELOG.md

SpriteAIMaker · Fit generated assets to the engine

Generated results still failed to match engine requirements. I separated the causes at each export stage and fixed them.

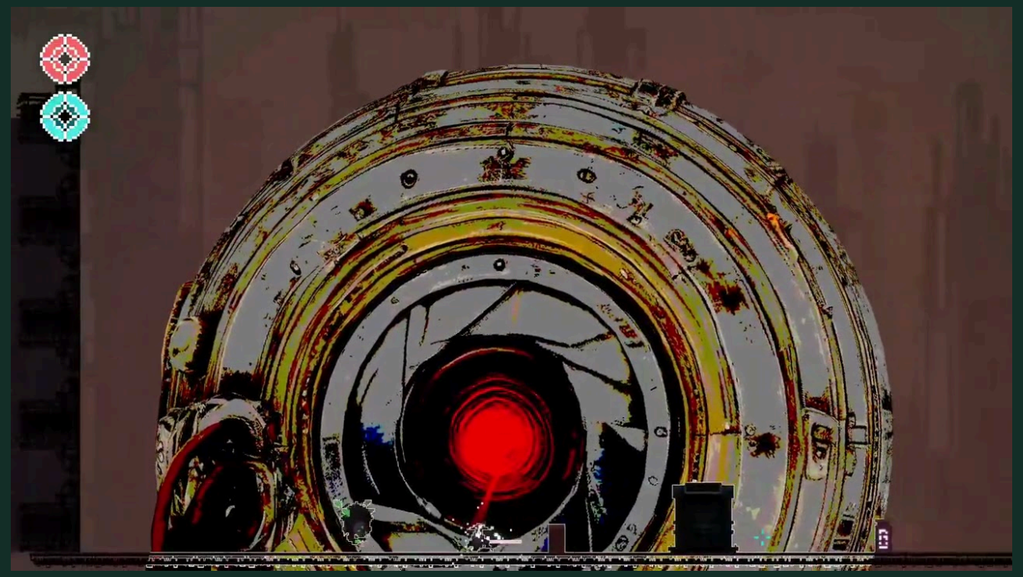
Export issue	Fix	Result
Duplicate first frame	Remove duplicate frames	Match the requested frame count
Different positions and sizes across frames	Normalize size and position around the bottom-center anchor	Align the motion reference point
Color postprocessing damaged pixel art	Avoid reducing the palette of finished pixel art again	Remove unnecessary reprocessing
Directions complete in a different order	Track requests and completion state per direction	Assemble four directions into one sheet

Connected production workflow

Review character → Generate animation → Process frames → Slice into individual images in Unity. The workflow continues until generated assets can be used in a game.

Issues and fixes ↗
github.com/kgr0831/SpriteAIMaker/blob/master/CHANGELOG.md

Remnents of Light · Development and release



Thumbnail from the Remnents of Light gameplay video

Download the game ↗ kgr0831.itch.io/remnents-of-light

Gameplay video ↗ youtube.com/watch?v=WHP7L8P5Be8

Problem found during development

Work depended on being at the development PC

Needed to request work and review results while away. Built ROLAI, connecting Discord, AI, and Unity to automate game development.

Project · Solo developer

2D action game · First-stage demo

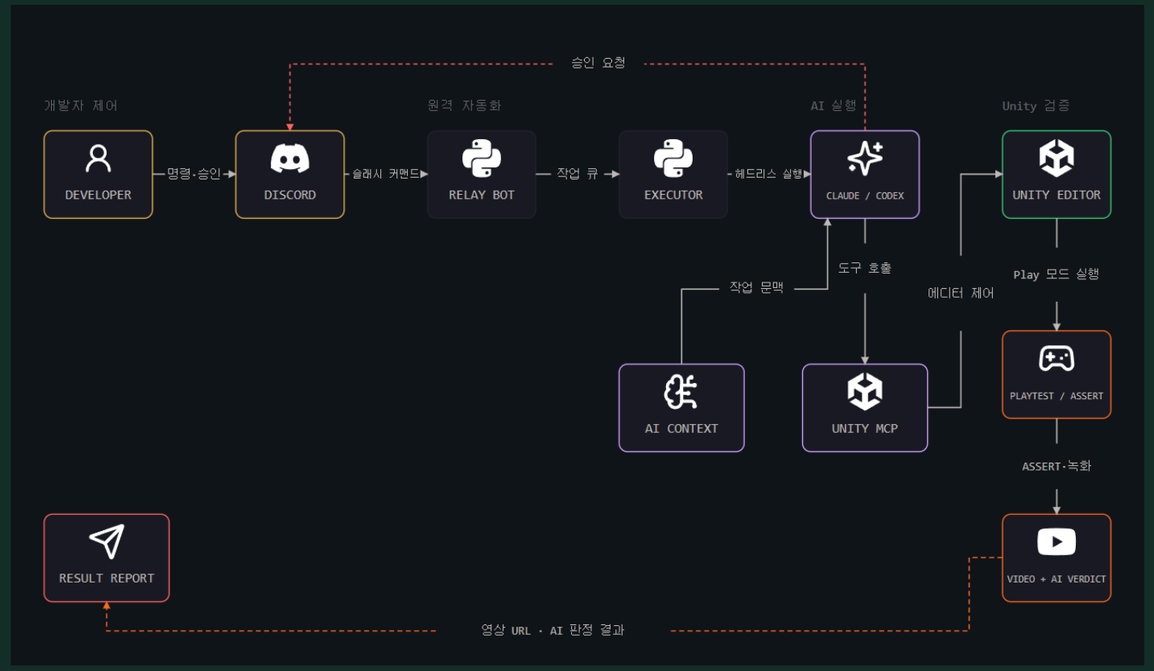
A Unity game where combos, parries, wall climbing, and dashes clear each room. Owned development, design, and art, and released a Windows demo on itch.io.

Next technical challenge

Beyond remote execution, establish validation criteria that AI can use to assess and revise its work.

Project code ↗
github.com/kgr0831/Remnents-of-Light

ROLAI · From remote requests to execution and reports



ROLAI architecture: requests, queue, execution, validation, and reporting

Work execution

Process Discord requests in order

The bot executes requests sequentially. AI operates the Unity Editor to implement and run changes, then reports results in Discord.

Validation workflow

Compare numeric logs with gameplay video

Check test conditions in logs and review actual behavior in short recordings. Revise and rerun based on the failure cause.

Implementation result

Built a remote development loop for requesting work and reviewing results away from the development PC.

Project code ↗
github.com/kgr0831/Remnents-of-Light

ROLAI · When the same test gives different results

Problem · Results depend on test conditions

The same test changed from run to run

Scene state, residual animation, and tuning values affected results, making it hard for AI to distinguish code defects from test-environment failures.

Solution

Fix conditions and classify failures

Start a separate test scene fresh each time. Distinguish setup failures from functional defects, and compare events and measurements before and after a fix.

Trade-off: video improves confidence but takes longer. Use logs for repeated checks and short recordings for final validation.



Discord view of development requests and validation reports

> **Result · Validation that distinguishes failure causes**
> Define pass conditions first so AI has a basis for repeated implementation and correction.

Project code ↗
github.com/kgr0831/Remnents-of-Light

ROLAI · Automation scope and validation cost



Runtime validation scene from the original project

What I learned

Defining pass conditions early mattered as much as describing features. Verifiable requirements provide a basis for repeated development.

Trade-off

AI autonomy and recoverable changes

AI repeats changes and validation within an allowed scope. Deletions, version-history changes, and game-scene changes require user approval to keep recovery manageable.

Trade-off

Confidence from video versus execution time

Logs, video, and AI assessment provide more evidence but take longer. Use fast log checks during iteration and short video checks at the end.

Initial setup and ongoing maintenance still cost time. The validation loop is not a guarantee of correctness.

Download the game ↗
kgr0831.itch.io/remnents-of-light

Project code ↗
github.com/kgr0831/Remnents-of-Light

Virtual Friend · A character conversation service

Web beta · Personal project · Business validation pending

Problem to explore · Across conversations, can a character still feel like the same person?

Developing a service with consistent character voice, appearance, and memory, addressing natural responses and continuity together.

Business hypothesis

Consistency and conversation quality may give users a reason to return. I plan to test which experiences they need, how they discover the service, and why they would pay.

To test: repeat use, character preferences, voice experience, operating cost, and willingness to pay



Character illustrations from the current prototype

> Alongside development, I explore **why people would use it and how it could become a business**.

Work in progress · Private ↗
github.com/kgr0831/vfriend

Early implementation · Calls and memory

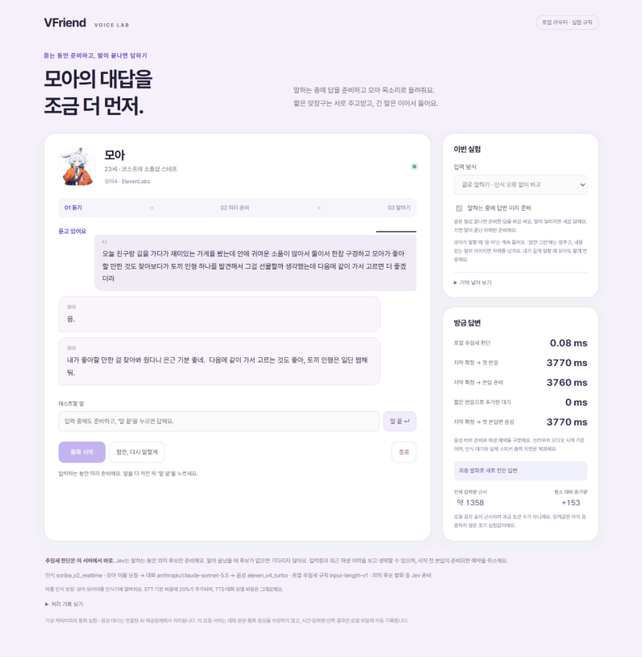
Incorrect turn detection cut conversations short. Saving unheard replies made the character's memory inconsistent.

Problem identified	Implementation	Early implementation
Cut-off speech and response delay	Automatic turn detection and interruptions; stream audio in the character's voice	Real-time calls in the local app Latency remains under development
Inconsistent call and chat memory	Save confirmed speech and played replies; carry call memories into later conversations	Memory storage and recovery Regression checks passed
Continuity of records and images	Per-account storage; generate and select images; retry failures and restore progress	Prototype implemented Checked with mock providers

> **Remaining work · Latency and visual consistency**
>
> One real-API trial with synthetic microphone input: 3.48s to first audio; 0.5s target unmet.
>
> Human microphone use and larger samples remain untested. Local LoRA training is future work.

Oct 25 research · Acknowledgments and interruptions

A problem beyond latency: if a simple “mm-hm” cuts off the reply, even a fast call feels unnatural.



Local voice research prototype · Scripted test input

User input	Turn-taking behavior
“Mm-hm · I see”	Keep the character speaking through acknowledgments
“Wait · Stop”	Cancel generation, speech synthesis and playback
“Uh” → “Yesterday...”	Transfer the turn without losing the full sentence

Testing reactions in both directions

Added brief listening reactions without ending the user’s turn. Recorded browser checks on Oct 25 covered acknowledgment, explicit stop, and long utterances.

Checks used real APIs with text input. This experiment used transcript rules; human microphone input and prosody still need evaluation.

Work in progress · Private ↗
github.com/kgr0831/vfriend

Virtual Friend · Prepare early, discard safely

Preparing an answer early may reduce waiting, but a correction can make that answer wrong before it is spoken.

Prepare while listening

Start a draft from a stable partial transcript

Check the final input

Compare text after whitespace and end-punctuation cleanup

Reuse or regenerate

Reuse an exact match; otherwise cancel and generate again

A quick reaction is not the answer

Cancel pending fillers when the answer is ready. Tested sentence grouping after tiny audio chunks broke intonation.

Recorded trial	Ready	Start
New answer from final input	2.053 s	2.063 s
Reuse the prepared answer	1.349 s	1.359 s
Regenerate after correction	1.883 s	1.893 s

One trial per path, real APIs with text input. Measured from final-transcript receipt to browser audio scheduling.
Excludes speech recognition and speaker output; not a controlled before/after test or a service guarantee.

Follow-up: account-call integration is on page 41. Semantic verification and human microphone evaluation remain open.

Virtual Friend · From voice lab to account calls

A voice lab’s session context could not carry a friendship forward. I connected account memory and character voices to the existing call screen.

Challenge	Implementation	Boundary checked
Context continuity	Load the character, recent chat and permitted account memories	Keep other friends’ memories separate
What the user heard	Record played phrases; extract memories after the call	Exclude unplayed reply fragments
Connection and failure	Check call availability and reuse mobile audio capture	Do not charge for a failed connection

Deployed web beta · Verified call continuity

An Oct 27 check used real APIs and a synthetic microphone: two replies played, recalled an earlier preference, and saved the call. On Oct 31, 21 related mock tests passed and public client files matched.

Applies to one-to-one calls. Group calls retain the prior engine. Human microphones, long calls and latency distributions remain unverified. Fillers do not measure answer speed.

Source · Private ↗
github.com/kgr0831/vfriend

Orca-STS · Voice conversations for development

Thinking aloud while reading code needs more than dictation. I extended Orca with continuous listening, spoken replies and turn handling.

Speech input
Local or selected speech provider

→

Codex conversation
Keep permissions and approvals

→

Streamed playback
Cancel queued audio on interruption

Challenge	My implementation	Scope
Turn decisions	Local models judge endpoints and overlap intent	Experimental synthetic-text evaluation
Recovery and cancellation	Discard stale decisions; retain uncertain sends	No automatic duplicate send or paid fallback

Windows preview released · Upstream contribution proposed

Released v1.4.214-sts.5 in my fork, with passing voice checks, typechecking and Windows build CI. The upstream PR for existing chats and terminals remains open.

The base IDE is existing Orca work. On Oct 31, 23 related mock tests passed. Hardware microphone quality, multilingual accuracy and faster end-to-end replies remain unproven.

Windows preview ↗
github.com/kgr0831/Orca-STS/releases/tag/v1.4.214-sts.5

Upstream contribution · Open ↗
github.com/stablyai/orca/pull/33850

Find the problem,
then choose how to solve it.

My role is to find problems.

Technology is a means to solve them.

kimgalam0831@gmail.com

github.com/kgr0831

kgr0831.github.io

Garam Kim